EEMS

PostgreSQL Database Blueprint

Relational design | Conceptual ERD | Executable starter DDL | RLS and migrations

IMPLEMENTATION-READY DESIGN BASELINE | VERSION 1.0

Audience: Solution architects, engineering leads, developers, QA, DevOps and project owners.

Scope: Multi-tenant election operations across parliamentary, upazila, union, municipality and city corporation elections.

Boundary: Administrative operations, compliance, personnel, budgets, center readiness, incident records and source-attributed results. No voter persuasion targeting or political-preference profiling.

Document control: Architecture baseline; adapt to current election law and obtain independent legal/security review before production.

Contents and delivery contract

Each numbered section is a development handoff. Deliver features with database migration, backend authorization, frontend permission guard, audit, tests and release notes. No section is production-complete until acceptance tests pass.

No.	Section
1	Database conventions and conceptual ERD
2	Schema catalog and ownership
3	Core executable DDL: tenants, geography, elections
4	Candidate and polling DDL
5	Financial and results schema DDL
6	RLS and security context
7	Indexes, constraints and integrity
8	Migration, seed and data lifecycle
9	Backup, HA, performance and acceptance
10	System invariants
11	Cross-team contracts

1. Database conventions and conceptual ERD

PostgreSQL 17+ compatible design; use a currently supported release at implementation time. Canonical geo data is shared, while election/candidate/operational data is tenant-owned. The relationship list below is the logical ERD.

```
core.tenants 1---N election.elections
core.tenants 1---N iam.tenant_users N---1 iam.users
election.election_types 1---N election.positions
geo.areas 1---N geo.areas (parent)
election.elections 1---N election.candidate_elections N---1 election.candidates
election.elections 1---N polling.election_centers N---1 polling.centers
election.elections 1---N team.teams 1---N team.team_members
election.elections 1---N finance.budgets 1---N finance.budget_lines
finance.budget_lines 1---N finance.expenses 1---N finance.expense_approvals
election.elections 1---N results.submissions 1---N results.details
results.submissions 1---N results.verifications
```

2. Schema catalog and ownership

Schema	Tables
core	tenants, tenant_settings, plans, subscriptions
iam	users, tenant_users, roles, permissions, role_permissions, user_roles, menus, user_scopes
geo	area_types, areas, constituencies, constituency_areas
election	election_types, positions, elections, candidates, candidate_elections, milestones
polling	centers, booths, election_centers, agent_assignments
team	teams, team_members, tasks, attendance, trainings
campaign	events, event_documents, venues, permits
finance	budget_categories, budgets, budget_lines, vendors, expenses, expense_approvals, payments
inventory	items, warehouses, stock_movements, allocations
operations	checklists, checklist_items, incidents, incident_reviews
results	submissions, details, verifications, official_results, result_documents
audit	audit_logs, security_events, export_logs, outbox_messages

3. Core executable DDL: tenants, geography, elections

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE SCHEMA IF NOT EXISTS core;
CREATE SCHEMA IF NOT EXISTS geo;
CREATE SCHEMA IF NOT EXISTS election;
CREATE TABLE core.tenants (
 tenant_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 code varchar(40) NOT NULL UNIQUE, name varchar(200) NOT NULL,
 status varchar(20) NOT NULL DEFAULT 'ACTIVE'
 CHECK (status IN ('ACTIVE','SUSPENDED','CLOSED')),
 created_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE geo.areas (
 area_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 parent_area_id bigint REFERENCES geo.areas(area_id),
 area_type varchar(30) NOT NULL, area_code varchar(40) NOT NULL,
 name_bn varchar(200) NOT NULL, name_en varchar(200),
 UNIQUE(area_type,area_code), CHECK(parent_area_id IS DISTINCT FROM area_id));
CREATE TABLE election.election_types (
 id smallint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 code varchar(40) NOT NULL UNIQUE, name varchar(120) NOT NULL);
CREATE TABLE election.positions (
 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 election_type_id smallint NOT NULL REFERENCES election.election_types(id),
 name varchar(120) NOT NULL, UNIQUE(election_type_id,name));
CREATE TABLE election.elections (
 election_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id uuid NOT NULL REFERENCES core.tenants(tenant_id),
 election_type_id smallint NOT NULL REFERENCES election.election_types(id),
 name varchar(200) NOT NULL, election_date date NOT NULL,
 area_id bigint NOT NULL REFERENCES geo.areas(area_id),
 status varchar(20) NOT NULL DEFAULT 'DRAFT'
 CHECK(status IN ('DRAFT','ACTIVE','CLOSED')),
 created_at timestamptz NOT NULL DEFAULT now(),
 UNIQUE(tenant_id,election_id));
CREATE INDEX ix_elections_tenant_date
 ON election.elections(tenant_id,election_date);
```

4. Candidate and polling DDL

```
CREATE SCHEMA IF NOT EXISTS polling;
CREATE TABLE election.candidates (
 candidate_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id uuid NOT NULL REFERENCES core.tenants(tenant_id),
 name varchar(200) NOT NULL, UNIQUE(tenant_id,candidate_id));
CREATE TABLE election.candidate_elections (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id uuid NOT NULL, election_id uuid NOT NULL,
 candidate_id uuid NOT NULL, position_id bigint NOT NULL
 REFERENCES election.positions(id), area_id bigint NOT NULL REFERENCES geo.areas(area_id),
 FOREIGN KEY(tenant_id,election_id)
 REFERENCES election.elections(tenant_id,election_id),
 FOREIGN KEY(tenant_id,candidate_id)
 REFERENCES election.candidates(tenant_id,candidate_id),
 UNIQUE(tenant_id,election_id,candidate_id,position_id));
CREATE TABLE polling.centers (
 center_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 code varchar(50) NOT NULL, name varchar(250) NOT NULL,
 area_id bigint NOT NULL REFERENCES geo.areas(area_id),
 UNIQUE(area_id,code));
CREATE TABLE polling.election_centers (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id uuid NOT NULL, election_id uuid NOT NULL,
 center_id uuid NOT NULL REFERENCES polling.centers(center_id),
 FOREIGN KEY(tenant_id,election_id)
 REFERENCES election.elections(tenant_id,election_id),
 UNIQUE(tenant_id,election_id,center_id));
```

5. Financial and results schema DDL

```
CREATE SCHEMA IF NOT EXISTS finance;
CREATE SCHEMA IF NOT EXISTS results;
CREATE TABLE finance.budgets (
 budget_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id uuid NOT NULL, election_id uuid NOT NULL,
 total_amount numeric(18,2) NOT NULL CHECK(total_amount>=0),
 FOREIGN KEY(tenant_id,election_id)
 REFERENCES election.elections(tenant_id,election_id),
 UNIQUE(tenant_id,budget_id));
CREATE TABLE finance.expenses (
 expense_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id uuid NOT NULL, budget_id uuid NOT NULL,
 amount numeric(18,2) NOT NULL CHECK(amount>0),
 status varchar(20) NOT NULL DEFAULT 'DRAFT'
 CHECK(status IN ('DRAFT','SUBMITTED','APPROVED','RETURNED')),
 created_at timestamptz NOT NULL DEFAULT now(),
 FOREIGN KEY(tenant_id,budget_id)
 REFERENCES finance.budgets(tenant_id,budget_id));
CREATE TABLE results.submissions (
 submission_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id uuid NOT NULL, election_id uuid NOT NULL,
 election_center_id uuid NOT NULL REFERENCES polling.election_centers(id),
 source_type varchar(30) NOT NULL, status varchar(20) NOT NULL
 DEFAULT 'DRAFT', submitted_at timestamptz,
 FOREIGN KEY(tenant_id,election_id)
 REFERENCES election.elections(tenant_id,election_id),
 UNIQUE(tenant_id,submission_id));
CREATE TABLE results.details (
 detail_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id uuid NOT NULL, submission_id uuid NOT NULL,
 candidate_election_id uuid NOT NULL REFERENCES election.candidate_elections(id),
 vote_count bigint NOT NULL CHECK(vote_count>=0),
 FOREIGN KEY(tenant_id,submission_id)
 REFERENCES results.submissions(tenant_id,submission_id));
```

Important: additional composite unique keys and composite foreign keys are required before production to ensure election_center_id and candidate_election_id belong to the same tenant AND election as the submission. These starter migrations are not the full production schema.

6. RLS and security context

```
ALTER TABLE election.elections ENABLE ROW LEVEL SECURITY;
ALTER TABLE election.elections FORCE ROW LEVEL SECURITY;
CREATE POLICY election_tenant_policy ON election.elections
USING (tenant_id = NULLIF(
 current_setting('app.tenant_id',true),'')::uuid)
WITH CHECK (tenant_id = NULLIF(
 current_setting('app.tenant_id',true),'')::uuid);
-- Inside a database transaction only, from verified server context:
SELECT set_config('app.tenant_id', :verified_tenant_id, true);
```

- Repeat RLS for every tenant-owned table; runtime DB role cannot own tables or BYPASSRLS.
- Fail closed if tenant context absent; pooled connections must not reuse previous request tenant state.
- Add election/area authorization in application and, where feasible, database policies.
- Audit privileged cross-tenant administrative actions separately.

7. Indexes, constraints and integrity

Object	Index / integrity
elections	(tenant_id,election_date); (tenant_id,status)
candidate_elections	(tenant_id,election_id,position_id); unique active assignment as business rules dictate
election_centers	unique(tenant_id,election_id,center_id)
expenses	(tenant_id,budget_id,status,created_at); invoice duplicate check
incidents	(tenant_id,election_id,center_id,occurred_at DESC)
results	(tenant_id,election_id,election_center_id,status)
audit_logs	(tenant_id,occurred_at DESC), partition by month at scale

• Use CHECK for local scalar rules, FK for references, unique constraints for identity and transactions for cross-row invariants.

• Use DEFERRABLE constraints only where justified; do not disable constraints during imports.

• Geo hierarchy cycle detection via trigger or application validator plus test.

• Optimistic concurrency with version integer or xmin strategy; document update semantics.

8. Migration, seed and data lifecycle

• EF Core migrations owned by backend modules; SQL scripts reviewed and applied by CI/CD release job.

• Seed election type codes: PARLIAMENTARY, UPAZILA, UNION, MUNICIPALITY, CITY; position mapping is configurable.

• Seed permission codes and default role templates; never seed production credentials.

• Import official geography and polling centers with provenance, effective date, checksum and review queue.

• Data retention per applicable law and contractual policy; archive closed elections with restore procedure.

• Use expand/migrate/contract for zero-downtime schema changes.

9. Backup, HA, performance and acceptance

Area	Implementation / validation
Backup	Nightly full + WAL archiving/PITR; verify restore
HA	Primary + standby if required; monitored failover procedures
Connection	Npgsql pool, PgBouncer if appropriate; transaction-mode RLS reviewed
Observability	slow query logs, pg_stat_statements, replication lag, WAL size
Security	least-privilege roles, TLS, encryption, audit, periodic access review
Testing	cross-tenant negative tests, concurrent approval, restore rehearsal

• Run EXPLAIN (ANALYZE, BUFFERS) against representative data before adding speculative indexes.

• Partition high-volume audit/events tables only when actual growth justifies complexity.

• Test that a result for center A cannot reference candidate/election from tenant B or election C.

• Before go-live, complete remaining schema migrations for IAM, teams, events, approvals, inventory, incidents, verification and audit.

10. System invariants

• Every tenant-owned row has tenant_id; all APIs derive tenant from authenticated context, never client input.

• An election has a type, schedule, geographic scope and lifecycle state. Position must belong to election type.

• Permissions are enforced in API and data access; hiding a menu is never authorization.

• Official results are clearly distinguished from unverified field submissions.

• Personal data is minimized; do not store inferred political preference or voter-level persuasion scores.

11. Cross-team contracts

Concern	Contract
Identifiers	UUID for tenant/election/candidate; bigint for canonical geographic codes where appropriate
Date/time	ISO 8601 UTC timestamps; election dates in local election timezone
API	/api/v1; RFC 7807 ProblemDetails; cursor/page pagination
State	Draft -> Submitted -> Reviewed -> Approved/Returned; record actor and time
Files	Object storage, malware scanning, SHA-256, signed downloads
Localization	Bangla/English display strings, stable English permission codes